

Megalomarket AI Core — Rapport de projet

Document de référence pour reprendre le projet à tout moment. Dernière mise à jour : 20 septembre 2026.

1. Résumé du projet

Objectif global

Megalomarket AI Core est un **backend central** qui connecte plusieurs canaux de vente (eBay, site propre, Amazon, TikTok Shop, Allegro) à une base de données unique, synchronise automatiquement stock et commandes, et utilise l'IA (Claude) pour assister la gestion quotidienne : prix, descriptions produit, support client, et **import de produits fournisseur par simple lien URL** (Alibaba, AliExpress...) avec génération automatique de fiches adaptées à chaque marketplace.

Ce n'est **pas** un ERP complet (pas de comptabilité, pas de multi-entrepôt, pas de facturation) — c'est un hub de synchronisation + moteur de décision IA.

État d'avancement actuel

Le service est **déployé et fonctionnel en production**, gratuitement, sans carte bancaire :

- **URL en ligne** : <https://megalomarket-ai-core.onrender.com>
- **Code source** : <https://github.com/sosodegra1-design/megalomarket> (branche `main`)
- **Hébergement** : Render (plan gratuit, déploiement automatique à chaque push sur `main`)
- **Base de données** : Turso (SQLite hébergé, gratuit, persistant — confirmé fonctionnel)
- **Tests** : 22 tests automatisés, tous passants, sans dépendance réseau ni vraie clé API

Fonctionnalités clés mises en place

1. **Synchronisation multicanal automatique** — stock (toutes les 15 min) et commandes

(toutes les 5 min), par canal, avec isolation des erreurs (un canal en panne ne bloque jamais les autres). 2.

Recommandations de prix par IA — analyse prix de revient + prix actuels + historique de ventes, avec refus automatique de toute suggestion sous le prix de revient. 3. **Génération de descriptions produit par IA**, adaptée au ton de chaque canal. 4. **Qualification de messages de support client par IA** — catégorie, urgence, réponse proposée — sans jamais décider seule d'un remboursement. 5. **Import de produit par lien fournisseur** (nouveau) :

- Extraction automatique (titre, description, prix d'achat, photos) depuis une URL.
- Génération IA de fiches adaptées à Amazon, TikTok Shop, Allegro et eBay en une requête.
- Calcul du prix de vente conseillé avec verrou anti-vente à perte.
- Deux modes : validation manuelle avant publication, ou publication automatique directe.

6. **Tableau de bord web basique** (<src/public/index.html>) pour visualiser canaux, produits et recommandations.

2. Architecture et code

Vue d'ensemble technique

Brique	Choix	Rôle
Serveur	Node.js + Express	API REST + fichiers statiques
Base de données	Turso (libSQL / SQLite hébergé)	Stockage persistant, gratuit
IA	Anthropic Claude (claude-sonnet-4-5)	Recommandations, génération, qualification
Hébergement	Render (plan free)	Exécution continue du serveur
CI locale	node --test (natif Node.js)	22 tests, aucune dépendance externe

Structure des dossiers

```
megalomarket/
|-- .env.example           # Modèle des variables d'environnement (jamais de vraies clés committées)
|-- .gitignore            # Exclut node_modules, .env, data/*.db, *.log
|-- package.json
|-- render.yaml           # Blueprint Render (service + variables secrètes)
|-- README.md             # Documentation d'installation et de déploiement
|-- Megapolemarket.md     # Ce document
|-- src/
|   |-- server.js         # Point d'entrée Express, init DB, démarre le planificateur
|   |-- config/
|   |   |-- env.js        # Lecture centralisée des variables d'environnement
|   |   |-- db/
|   |       |-- database.js # Client Turso + helpers dbAll/dbGet/dbRun + logActivity
|   |       |-- schema.sql  # Définition de toutes les tables
|   |-- connectors/       # Un fichier par canal, interface commune
|   |   |-- index.js       # Registre des connecteurs + statut actif/inactif
|   |   |-- ebay.js        # OAuth2 + Sell API – SEUL connecteur pleinement actif
|   |   |-- ownSite.js     # Contrat REST générique (hypothèse à valider avec le vrai site)
|   |   |-- amazon.js     # Squelette – en attente d'approbation SP-API
|   |   |-- tiktokShop.js  # Squelette – en attente d'approbation Partner API
|   |   |-- allegro.js     # Squelette – en attente d'inscription développeur
|   |-- ai/
|   |   |-- client.js      # Appel générique à l'API Anthropic
|   |   |-- priceOptimizer.js # Recommandations de prix
|   |   |-- descriptionWriter.js # Génération de descriptions par canal
|   |   |-- supportAgent.js # Qualification des messages de support
|   |-- importer/         # Module d'import produit par lien (nouveau)
|   |   |-- scraper.js     # Extraction JSON-LD / meta-tags / repli HTML
|   |   |-- pricing.js     # Calcul du prix conseillé (fonction pure, testée isolément)
|   |   |-- listingGenerator.js # Génération IA multi-marketplaces
|   |   |-- publisher.js   # Publication via le connecteur de la marketplace choisie
|   |-- services/
|   |   |-- stockSync.js   # Synchronisation stock, tous canaux actifs
|   |   |-- orderSync.js  # Synchronisation commandes, tous canaux actifs
|   |   |-- scheduler.js   # Tâches planifiées (node-cron)
|   |-- routes/
|   |   |-- api.js         # Routes principales (produits, commandes, recommandations...)
|   |   |-- imports.js     # Routes du module d'import produit
|   |-- public/
|   |   |-- index.html     # Tableau de bord web
|-- tests/
|   |-- env.test.js
|   |-- database.test.js
|   |-- connectors.test.js
|   |-- pricing.test.js
|   |-- scraper.test.js
```

Schéma de base de données (Turso / SQLite)

```
-- Catalogue interne
products (id, sku UNIQUE, name, description, cost_price, created_at)
channel_listings (id, product_id, channel, external_id, price, description, stock, status, updated_at, UNIQUE(pr
orders (id, channel, external_order_id, product_id, quantity, amount, status, created_at, UNIQUE(channel, extern
```

```
recommendations (id, type, channel, product_id, payload JSON, status, created_at)
activity_log (id, kind, message, created_at)

-- Module d'import produit
imports (id, source_url, source_site, title, raw_description, purchase_price, currency, image_urls JSON, status,
import_listings (id, import_id, marketplace, title, description, suggested_price, status, published_external_id,
```

Blocs de configuration essentiels

Connexion à la base de données (`src/db/database.js`) — le même client fonctionne en local (fichier SQLite) et en production (Turso), seule l'URL change :

```
import { createClient } from '@libsql/client';
import { config } from '../config/env.js';

export const client = createClient({
  url: config.turso.url, // "file:./data/megalomarket.db" en local, "libsql://..." en prod
  authToken: config.turso.authToken,
});
```

Calcul du prix conseillé avec verrou anti-perte (`src/importer/pricing.js`) :

```
export function computeSuggestedPrice(purchasePrice, { marginCoefficient = 1.8, fixedFee = 0 } = {}) {
  const floor = purchasePrice + fixedFee;
  const raw = purchasePrice * marginCoefficient + fixedFee;
  return Math.round(Math.max(raw, floor) * 100) / 100; // jamais sous le prix de revient
}
```

Blueprint Render (`render.yaml`) — plan gratuit, aucune carte requise, secrets injectés depuis le dashboard Render (jamais commités) :

```
services:
- type: web
  name: megalomarket-ai-core
  runtime: node
  plan: free
  buildCommand: npm install
  startCommand: npm start
  envVars:
    - key: TURSO_DATABASE_URL
      sync: false
    - key: TURSO_AUTH_TOKEN
      sync: false
    - key: ANTHROPIC_API_KEY
      sync: false
  # + clés eBay, Amazon, TikTok Shop, Allegro (toutes sync: false)
```

API — endpoints principaux

GET /api/health	Vérification de disponibilité
GET /api/channels	Statut de chaque connecteur (actif/inactif)
GET /api/products	Liste des produits + leurs fiches par canal
POST /api/products	Créer un produit {sku, name, description?, costPrice?}
GET /api/orders	Commandes récentes
POST /api/sync/orders	Déclenche la synchro commandes manuellement
POST /api/sync/stock	Déclenche la synchro stock manuellement
GET /api/recommendations?status=pending	Recommandations IA (prix/description/support)
POST /api/recommendations/price/:productId	Générer des suggestions de prix
POST /api/recommendations/description/:productId/:channel	Générer une description
POST /api/recommendations/:id/apply	Marquer une recommandation comme appliquée
POST /api/recommendations/:id/dismiss	Rejeter une recommandation
POST /api/support/qualify	Qualifier un message client {orderId?, customerMessage?}
GET /api/activity	Journal d'activité

# --- Module d'import produit ---	
GET /api/imports	Liste des imports
POST /api/imports	{url} – extraction depuis une page fournisseur
GET /api/imports/:id	Détail d'un import + ses fiches par marketplace
POST /api/imports/:id/generate	Génération IA des fiches + prix conseillé
PATCH /api/imports/:id/listings/:marketplace	Option A : corriger une fiche avant publication
POST /api/imports/:id/listings/:marketplace/publish	Option B : publier directement (eBay seul actif)

3. Commandes et procédures

Installation locale

```
git clone https://github.com/sosodegrai-design/megalomarket.git
cd megalomarket
npm install
cp .env.example .env
# Remplir .env avec les vraies clés (minimum ANTHROPIC_API_KEY pour tester l'IA)
```

Lancer en local

```
npm run dev      # avec rechargement automatique (node --watch)
# ou
npm start       # sans rechargement automatique
```

Le tableau de bord est servi sur `http://localhost:3000`.

Tests

```
npm test      # exécute les 22 tests (node --test), sans réseau ni vraie clé API
```

Déploiement

Le déploiement est **automatique** : tout `git push` sur la branche `main` déclenche un redéploiement sur Render (Blueprint synchronisé au dépôt GitHub).

```
git add -A
git commit -m "message"
git push origin main
```

Pour ajouter/modifier une clé API en production : Render → service `megalomarket-ai-core` → menu

Environment → éditer la variable → **Save Changes** (redémarre automatiquement).

Créer une base Turso (si besoin d'en recréer une)

```
turso db create megalomarket
turso db tokens create megalomarket
turso db show megalomarket --url
```

Exemple d'appel à l'API en production

```
curl https://megalomarket-ai-core.onrender.com/api/health
curl -X POST https://megalomarket-ai-core.onrender.com/api/imports \
  -H 'Content-Type: application/json' \
  -d '{"url":"https://www.aliexpress.com/item/XXXXXXX.html"}'
```

4. Tâches restantes et pistes d'amélioration

Bloquant / en attente côté utilisateur

[] `ANTHROPIC_API_KEY` — sans elle, toutes les fonctionnalités IA (prix, descriptions, support, import produit) renvoient une erreur claire mais restent inactives.

[] **Clés Amazon SP-API** — en attente d'approbation développeur.

[] **Clés TikTok Shop Partner API** — en attente d'approbation.

[] **Clés Allegro** — nécessite une inscription développeur sur `apps.developer.allegro.pl`.

[] **Confirmer la vraie architecture de** `bbhappy.onrender.com` (le site existant) pour finaliser `src/connectors/ownSite.js`, actuellement basé sur une hypothèse de contrat REST (`GET /products`, `POST /products/:sku/price`, `POST /products/:sku/stock`).

Limites connues (pas des bugs — des contraintes structurelles)

- **Scraping Alibaba/AliExpress non garanti à 100%** — ces sites bloquent activement les requêtes automatisées ou chargent leur contenu en JavaScript. L'extraction échoue alors proprement avec un message clair, mais certaines fiches devront être complétées à la main.
- **Le plan gratuit Render se met en veille après ~15 min d'inactivité**, ce qui retarde la première requête après une pause et interrompt les synchronisations planifiées pendant la veille. Solution recommandée : configurer un ping externe gratuit (ex. `cron-job.org`) vers `/api/health` toutes les 10-14 minutes.
- **Publication automatique limitée à eBay** pour l'instant (seul canal avec clés actives) ; Amazon/TikTok Shop/Allegro suivent la même interface mais restent inactifs tant que leurs clés ne sont pas fournies.
- **eBay `createListing` nécessite un compte vendeur avec ses "business policies" configurées** (paiement, livraison, retours) — sans quoi l'API eBay refusera la publication avec un message d'erreur explicite (déjà remonté tel quel par le code).

Améliorations possibles (non commencées, à prioriser ensemble)

- Tableau de bord web pour le module d'import (aujourd'hui accessible uniquement via l'API).
- Authentification sur le tableau de bord avant toute exposition publique plus large.
- Alertes de stock bas, historisation des prix, export CSV/Excel des ventes.
- Implémentation réelle des connecteurs Amazon, TikTok Shop et Allegro dès obtention des clés (le contrat d'interface — `listOrders`, `listInventoryItems`, `updateOfferPrice`, `createListing`, `isConfigured` — est déjà en place, aucun autre fichier n'aura besoin de changer).
- Gestion multi-entrepôt / multi-fournisseur si le volume le justifie un jour.

Historique des étapes clés (chronologique)

1. Suppression complète de l'ancien projet (Marketplace B2B) à la demande explicite de l'utilisateur, pour repartir de zéro sans rien mélanger à ses fichiers existants. 2. Construction initiale de Megalomarket AI Core (connecteurs, IA, base de données, tableau de bord, tests) avec SQLite local (`better-sqlite3`). 3. Création du dépôt GitHub `sosodegra1-design/megalomarket` et envoi du code. 4. Tentative de déploiement sur Render → carte bancaire demandée (plan payant, disque de stockage nécessaire pour SQLite). 5. Migration de la base de données vers **Turso** (SQLite hébergé, gratuit, sans carte) — remplacement de `better-sqlite3` par `@libsql/client`, toutes les requêtes converties en asynchrone. 6. Déploiement réussi sur Render en **plan gratuit** — service en ligne, base persistante vérifiée (écritures confirmées côté Turso). 7. Ajout du **module d'import produit par lien** : scraping, génération IA multi-marketplaces, calcul de prix sécurisé, connecteur eBay étendu (publication réelle), connecteur Allegro (nouveau, en attente), routes Express complètes pour les deux modes (validation manuelle / publication automatique).